



YENEPOYA (DEEMED TO BE UNIVERSITY)

Project Synopsis on **AI SELF-HEALING CLOUD FOR RETAIL**

Student Name:

Mohammed Raeed (23BCAICD064)

Industry Mentor: Mr. Sumit Kumar Shukla

Guided by: Mrs. Sindhu

TABLE OF CONTENTS

Contents	Page No.
1. INTRODUCTION	3
2. PROBLEM STATEMENT	4
3. OBJECTIVES	5
4. METHODOLOGY	6
5. SYSTEM REQUIREMENTS	7
6. FUTURE SCOPE	8
7. CONCLUSION	9

1. INTRODUCTION

The modern retail landscape is fundamentally defined by its reliance on sophisticated, real-time software systems to facilitate transactions and inventory management. Point-of-Sale (POS) infrastructure represents the critical lifeblood of business operations, acting as the primary touchpoint between the enterprise and the consumer. However, as retail applications grow in complexity—integrating multiple APIs, payment gateways, and inventory trackers—they become increasingly susceptible to software-level instabilities. Issues such as unhandled exceptions, memory exhaustion, and thread lockups can cripple a checkout lane in seconds. This project, "AI Self-Healing Cloud for Retail," proposes a robust, autonomous Service Reliability Engineering (SRE) framework designed to solve this fragility. It utilizes a lightweight Python-based daemon to monitor application states on edge hardware, ensuring immediate recovery from software failures without the need for manual technical intervention.

By bringing cloud-native automation principles down to the individual kiosk, this system ensures that even a catastrophic crash in the POS software does not lead to significant downtime. The application acts as an intelligent watchdog, constantly verifying that critical processes are active and responsive. This approach is not just about keeping the software running; it is about protecting the financial throughput of the retail store and maintaining a seamless experience for the customer. The result is a high-availability infrastructure that functions as an autonomous edge-cloud node, resilient against both local application errors and intermittent environmental glitches.

2. PROBLEM STATEMENT

Downtime in retail environments causes immediate financial leakage and irreparable damage to the customer experience. Currently, the resolution of Point-of-Sale (POS) software crashes relies heavily on manual intervention by checkout staff or remote IT support teams. This manual troubleshooting process is characterized by significant latency. A cashier must first identify that the application has frozen, alert a store manager, and then either attempt a hard reboot or wait for a support ticket to be resolved. During this window, which can span from several minutes to hours, checkout lines grow stagnant, frustrated customers abandon their purchases, and the store suffers direct revenue loss.

Furthermore, most retail edge devices operate in a technical vacuum, lacking the sophisticated automated orchestration layers found in enterprise-grade data centers. While tools like Kubernetes provide self-healing for cloud-hosted microservices, standalone retail terminals are often left vulnerable without any automated safety net. There is an urgent, critical need for a lightweight, localized 'self-healing' mechanism that can provide enterprise-level high availability on standard, standalone retail terminals. This project addresses the gap between enterprise cloud reliability and localized hardware reality, providing a cost-effective automation layer that safeguards the retail environment from the costs of technical failure.

3. OBJECTIVES

The primary technical and operational objectives of this project are designed to transform reactive maintenance into proactive automation. To achieve the goals of Project 36, the system focuses on the following specific milestones:

- **High-Performance Autonomous Monitoring:** To engineer a background daemon utilizing the native Python *psutil* library. This daemon will scan operating system process trees in real-time with sub-1% CPU overhead, ensuring the monitoring tool does not cannibalize hardware resources needed by the POS application.
- **Deterministic Self-Healing Protocols:** To implement an execution logic that instantly detects unexpected application terminations or 'Not Responding' states. Upon detection, the system will trigger a forceful sub-process recovery via the *subprocess* module, relaunching the critical application in its last known healthy state within milliseconds.
- **Relational Data Persistence:** To facilitate immutable data logging through an ACID-compliant SQLite3 database. The system will simulate and record retail transaction data (receipts, timestamps, and amounts) to prove that the ledger remains consistent and secure even across multiple aggressive application reboots.
- **Real-Time Observability and Telemetry:** To develop a decoupled telemetry interface using *Flask-SocketIO*. This will stream operational metrics, shift totals, and system health status to a localized dashboard, allowing administrators to monitor the health of their hardware fleet without manual reporting.

4. METHODOLOGY

The architecture of the AI Self-Healing Cloud utilizes an advanced Asynchronous Multi-Threaded approach to ensure absolute operational decoupling and system stability. The software is constructed as a monolithic Python script for ease of deployment, but is logically split into two primary execution pathways. The **Primary Thread** hosts the WSGI Flask Server and manages the lifecycle of full- duplex WebSocket handshakes. This thread remains dedicated to networking, ensuring that store managers can always access the health dashboard regardless of how hard the backend is working.

Concurrently, a **Secondary Daemon Thread** operates the infinite monitoring and simulation loop. This thread is configured with the *daemon=True* flag to ensure it is safely terminated by the OS when the host process stops. Within this loop, the script continuously queries the host kernel for target process IDs (e.g., the POS executable). If the process is missing, it triggers the healing protocol. Following the health check, the thread generates stochastic retail transaction data using Python's random module to simulate a live environment. This data is then simultaneously committed to the SQLite disk file and pushed into the shared memory state. Finally, the updated state is broadcasted to the frontend via WebSockets, allowing the Chart.js library to render live updates on the dashboard canvas.

5. SYSTEM REQUIREMENTS

5.1 Hardware Requirements

The system is specifically engineered for edge deployment on localized retail hardware, requiring minimal overhead to maintain maximum performance. The minimum specifications include a standard x86/x64 or ARM-based processor (2.0 GHz dual-core recommended) and at least 4GB of system RAM. These resources are sufficient to host both the Python monitoring daemon and the primary retail applications simultaneously. For data integrity, a standard Solid State Drive (SSD) is required to ensure low-latency relational database commits during rapid automation cycles. No specialized server infrastructure or external cloud instances are necessary, as the terminal functions as a fully autonomous edge node.

5.2 Software Requirements

The host environment must execute a modern desktop operating system such as Windows 10/11 or a standard Linux distribution. The application requires a Python 3.8+ runtime environment. Core library dependencies include *Flask* for the web backend, *Flask-SocketIO* for telemetry streaming, *psutil* for cross-platform OS process management, and *pyautogui* for simulating human hardware interactions. To ensure the self-healing daemon remains active and starts automatically upon boot, it must be granted administrative privileges through the host's native Task Scheduler or Cron services.

6. FUTURE SCOPE

While the current implementation relies on deterministic recovery logic (relaunching an app once it has already failed), the future scope of this project involves moving from reactive to **Predictive Self-Healing**. By integrating Machine Learning classifiers—such as Random Forest or LSTM models—the system could analyze historical telemetry regarding CPU thermal spikes and memory utilization trends to predict an impending software crash **before** it occurs. This would allow the system to gracefully reboot the application during a quiet period in the store, achieving true zero-downtime operations.

Additionally, the project can be expanded into a **Distributed Fleet Management** system. By containerizing the monitoring daemon using Docker, the architecture could be deployed across thousands of globally distributed retail franchises. A centralized cloud control plane would then aggregate telemetry from every individual edge-node, providing corporate headquarters with a real-time global overview of their IT infrastructure health and total revenue throughput without requiring on-site staff to submit any reports.

7. CONCLUSION

The "AI Self-Healing Cloud for Retail" project successfully demonstrates that high-end Service Reliability Engineering (SRE) can be effectively downscaled for localized retail environments. By replacing manual troubleshooting with an intelligent, autonomous Python script, the system drastically reduces Mean Time to Recovery (MTTR) from several minutes down to a fraction of a second. This ensures that Point-of-Sale operations remain continuous, safeguarding the financial throughput of the business and maintaining a superior customer experience. This project serves as a scalable and modern blueprint for the future of decentralized IT operations, proving that software automation at the edge is the most efficient method for maintaining high availability in the retail sector.